SIMATS SCHOOL OF ENGINEERING

Approved by AICTE | IET-UK Accreditation

Course Code: DSA03	**Course Name: Natural Language Processing**
Branch: CSE	**Year: 2025**
Name: Rakshitha V B	**Reg No: 192324004**
Date: 05/11/2026	**Dept: AI&DS**

Assignment 1

Question 1: FSA for Absolute Date Expressions

The automaton below accepts absolute dates such as **March 15**, **the 22nd of November** and **Christmas**. Deictic expressions that depend on the current day (for example "the day before yesterday") are deliberately not accepted. Every edge carries a word or a class of words, and the shorthand classes listed in the table keep the diagram small. The start state is q0, the double-circled states (q3, q5, q9, q10, q11, q12) are accepting, and the full diagram is on the next page.

Shorthand classes used on the edges

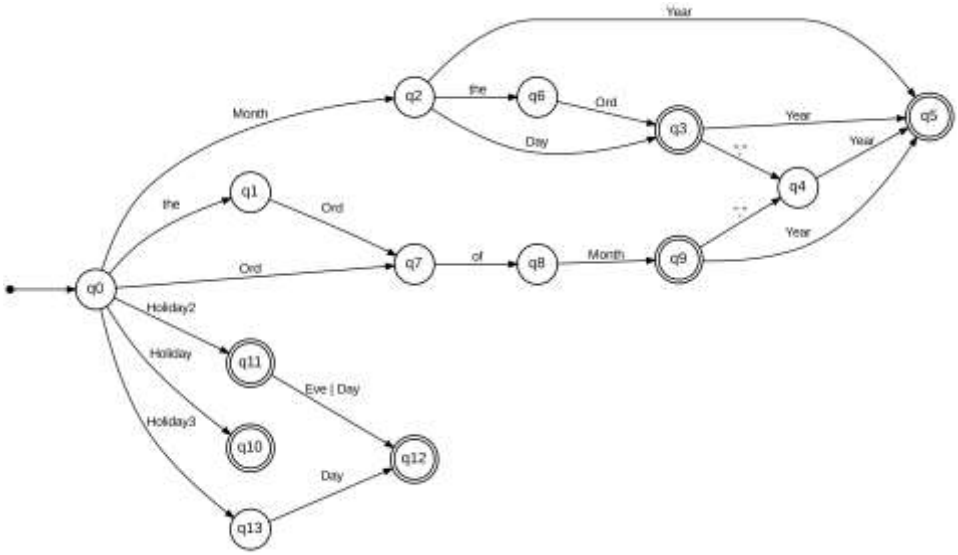
Class	Words in the class
Month	January, February, ..., December and the abbreviations Jan, Feb, Mar, Apr, Jun, Jul, Aug, Sep, Sept, Oct, Nov, Dec
Day	1, 2, 3, ..., 31 and the written ordinals 1st, 2nd, 3rd, ..., 31st
Ord	1st, 2nd, 3rd, 4th, ..., 31st and first, second, third, ..., thirty-first
Year	any four-digit number such as 1999 or 2025
Holiday	Easter, Halloween, Thanksgiving, Diwali, Holi, Pongal
Holiday2	Christmas, New Year's (may be followed by Eve or Day)
Holiday3	Independence, Valentine's, Labor, Memorial, Republic (must be followed by Day)

Strings accepted and the paths they take

String	Path through the FSA
March 15	q0 -Month-> q2 -Day-> q3
March 15, 2025	q0 -Month-> q2 -Day-> q3 -","-> q4 -Year-> q5
March the 15th	q0 -Month-> q2 -the-> q6 -Ord-> q3
the 22nd of November	q0 -the-> q1 -Ord-> q7 -of-> q8 -Month-> q9
Christmas	q0 -Holiday2-> q11
Christmas Eve	q0 -Holiday2-> q11 -Eve-> q12
Independence Day	q0 -Holiday3-> q13 -Day-> q12

Question 1: FSA Diagram

Figure 1: FSA recognising absolute date expressions (double circles are accepting states)



Question 2: Regular Expressions

Notation used: Perl (PCRE) notation. A "word" is a run of letters. The word-boundary anchor `\b` treats white space, line breaks and punctuation as separators, so punctuation next to a word never becomes part of it. Each pattern below was run on sample strings, and the output is shown in the box beneath it.

(a) The set of all alphabetic strings

One or more letters, anchored at both ends so that no digit, space or punctuation mark is allowed.

```
^[A-Za-z]+$
```

```
Pattern: ^[A-Za-z]+$

'Language'                    MATCH
'NLP2025'                     NO MATCH
'hello world'                 NO MATCH
''                            NO MATCH
'data-set'                    NO MATCH
```

Output for (a)

(b) All lowercase alphabetic strings ending in b

Any number of lowercase letters followed by a final b, anchored at both ends.

```
^[a-z]*b$
```

```
Pattern: ^[a-z]*b$

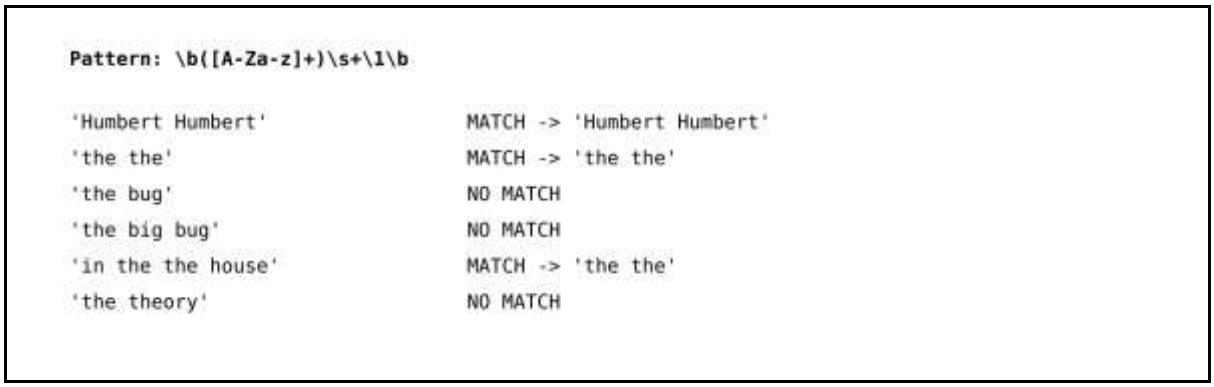
'club'                        MATCH
'crab'                        MATCH
'b'                           MATCH
'Club'                        NO MATCH
'bobs'                        NO MATCH
'cab1'                        NO MATCH
```

Output for (b)

(c) All strings with two consecutive repeated words

The parentheses capture a word and the back-reference `\1` demands the same word again after white space. The boundaries `\b` at both ends stop partial matches, so "the theory" is rejected, while "Humbert Humbert" and "the the" match. Adding `(?i)` in front makes the comparison case-insensitive.

```
\b([A-Za-z]+)\s+\1\b
```

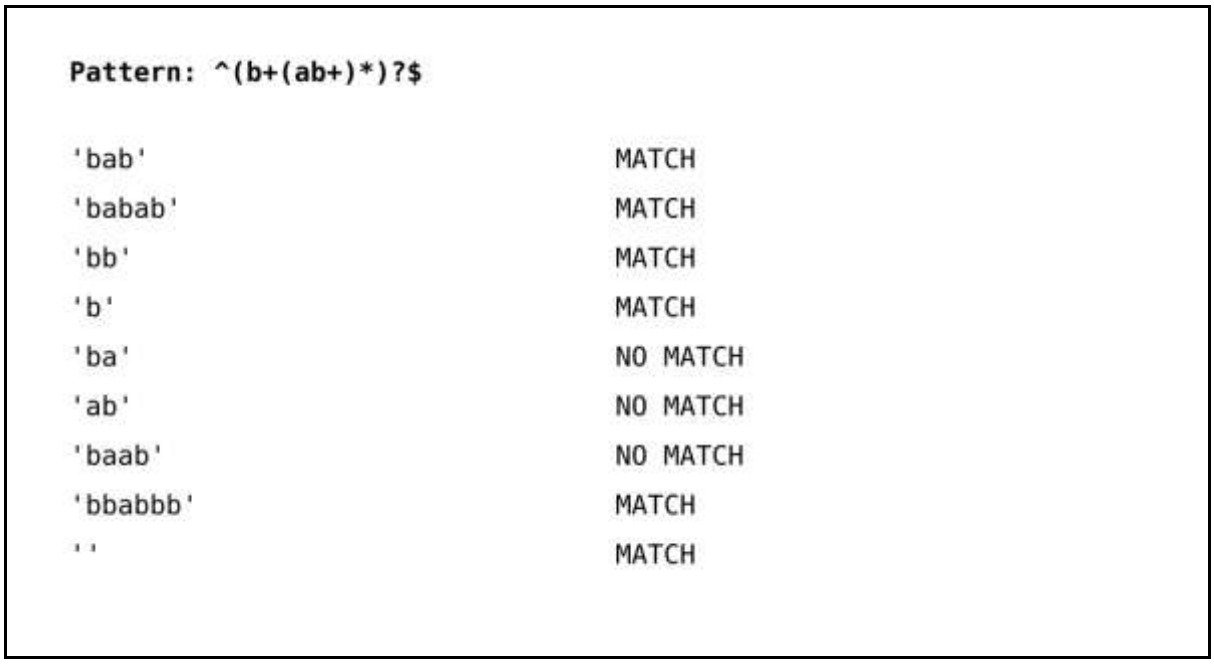


Output for (c)

(d) Strings over {a, b} where every a is immediately preceded and followed by b

The string is either empty or starts with b, and every a sits between b's. Writing the string as a block of b's followed by any number of (a, block of b's) pairs enforces this, so "aa", a leading a and a trailing a are all impossible.

```
^(b+(ab+)*)?$
```

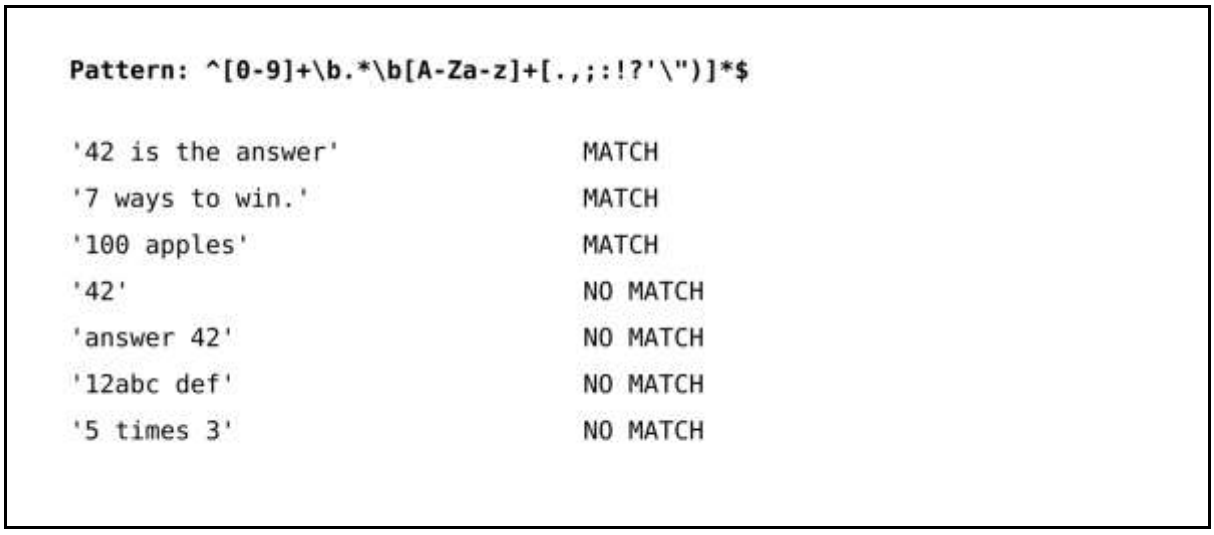


Output for (d)

(e) Lines that start with an integer and end with a word

The line begins with digits followed by a word boundary, so "12abc" is rejected. The middle is arbitrary, and the line ends with a word, optionally followed by closing punctuation such as a full stop or a quotation mark.

```
^[0-9]+\b.*\b[A-Za-z]+[.,;:!?'")]*$
```



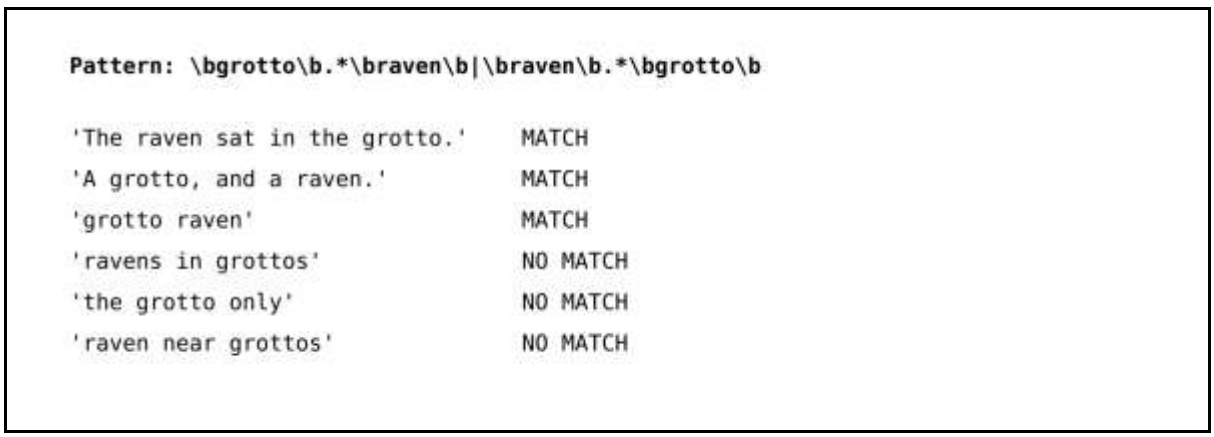
Output for (e)

(f) Strings containing both the word grotto and the word raven

Both words are matched with `\b` on each side, so "grottos" or "ravens" do not count. Because either word may come first, the two orders are joined with alternation. An equivalent form uses look-aheads.

```
\bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b

^(?=.*\bgrotto\b)(?=.*\braven\b)
```



Output for (f)

(g) A pattern that places the first word of an English sentence in a register

The register is capture group 1 (`\1`). The pattern starts either at the beginning of the text or after sentence-ending punctuation (. ! ?) with optional closing quotes or brackets and white

space. It then skips opening quotes or brackets and captures a word that may contain internal apostrophes or hyphens, so "Don't" is captured whole.

```
(?x)
(?: ^ | [.!?]["')\]]* \s+ )
["'(\[]*
( [A-Za-z]+ (?: ['’-] [A-Za-z]+ )* )
```

```
Pattern: capture group 1 = first word of each sentence

Input : The cat sat. "Dogs bark!" Yes, they do. (Maybe) it rains. Don't stop now.
Register \1 values, in order:
    1. The
    2. Dogs
    3. Yes
    4. Maybe
    5. Don't
```

Output for (g)

Question 3: Minimum Edit Distance

Version used: insertion = 1, deletion = 1, substitution = 2. The table is filled with the recurrence D[i, j] = min(D[i-1, j] + 1, D[i, j-1] + 1, D[i-1, j-1] + c), where c = 0 if the two letters are equal and 2 otherwise. The source is **drive**, written down the side, and the target is written along the top. Shaded cells mark the optimal alignment path.

The same computation can be checked with this program.

```
def edit_distance(source, target):
    n, m = len(source), len(target)
    D = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        D[i][0] = D[i - 1][0] + 1
    for j in range(1, m + 1):
        D[0][j] = D[0][j - 1] + 1
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = 0 if source[i - 1] == target[j - 1] else 2
            D[i][j] = min(
                D[i - 1][j] + 1,
                D[i][j - 1] + 1,
                D[i - 1][j - 1] + cost
            )
    return D[n][m]

print(edit_distance("drive", "brief"))
print(edit_distance("drive", "divers"))
```

drive to brief

drive -> brief distance = 4

ins = 1, del = 1, sub = 2

	#	b	r	i	e	f
#	**0**	1	2	3	4	5
d	1	**2**	3	4	5	6
r	2	3	**2**	3	4	5
i	3	4	3	**2**	3	4
v	4	5	4	**3**	4	5
e	5	6	5	4	**3**	**4**

Distance matrix for drive and brief

Alignment (s = substitution, d = deletion, i = insertion, | = match):

```
source : d  r  i  v  e  *
target : b  r  i  *  e  f
ops    : s  |  |  d  |  i
cost   : 2  0  0  1  0  1    total = 4
```

drive to divers

drive -> divers distance = 3

ins = 1, del = 1, sub = 2

	#	d	i	v	e	r	s
#	**0**	1	2	3	4	5	6
d	1	**0**	1	2	3	4	5
r	2	**1**	2	3	4	3	4
i	3	2	**1**	2	3	4	5
v	4	3	2	**1**	2	3	4
e	5	4	3	2	**1**	**2**	**3**

Distance matrix for drive and divers

Alignment:

```
source : d  r  i  v  e  *  *
target : d  *  i  v  e  r  s
ops    : |  d  |  |  |  i  i
cost   : 0  1  0  0  0  1  1    total = 3
```

Conclusion

drive is closer to divers. The edit distance from drive to divers is **3**, while the edit distance from drive to brief is **4**. (With every operation costing 1, both words would tie at 3, so the substitution cost of 2 is what separates them.)

```
Comparison

distance(drive, brief ) = 4
distance(drive, divers) = 3

Closest word to 'drive' : divers
```

Comparison of the two distances